Functions

ISTITUTO TECNOLOGICO SUPERIORE ACADEMY - INFORMATION AND COMMUNICATIONS TECHNOLOGY
its
ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Exercise

Suppose that you need to find the sum of integers from **1** to **10**, **20** to **37**, and **35** to **49**. Write a Python program that compute these three different sums.

<u>Expected Output</u>:

Sum of integers from 1 to 10 is **55**

Sum of integers from 20 to 37 is **513**

Sum of integers from 35 to 49 is **630**

ISTITUTO TECNOLOGICO SUPERIORE ACADEMY - INFORMATION AND COMMUNICATIONS TECHNOLOGY
its ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Solution

If you create a program to add these three sets of numbers, your code might look like this:

```
# sum of integers from 1 to 10
sum:int = 0
for i in range(1, 11):
      sum += i
print("Sum from 1 to 10 is", sum)

# sum of integers from 20 to 37
sum:int = 0
for i in range(20, 38):
      sum += i
print("Sum from 20 to 37 is", sum)

# sum of integers from 35 to 49
sum:int = 0
for i in range(35, 50):
      sum += i
print("Sum from 35 to 49 is", sum)
```

its ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Functions - Introduction

You may have observed that the code for computing these sums is very similar, except that the starting and ending integers are different.

Wouldn’t it be nice to be able to write commonly used code once and then reuse it?

You can do this by defining a function, which enables you to create reusable code.

ISTITUTO TECNOLOGICO SUPERIORE ACADEMY - INFORMATION AND COMMUNICATIONS TECHNOLOGY
its
ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Defining a Function/1

The syntax for defining a function is as follows:

```
def functionName(list of parameters):
        instructions for function body
```

ISTITUTO TECNOLOGICO SUPERIORE ACADEMY - INFORMATION AND COMMUNICATIONS TECHNOLOGY
its
ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Exercise

We can rewrite this code simplifying it using functions.

Suppose that you need to find the sum of integers from **1** to **10**, **20** to **37**, and **35** to **49**. Write a Python program that compute these three different sums.

<u>Expected Output</u>:

Sum of integers from 1 to 10 is **55**

Sum of integers from 20 to 37 is **513**

Sum of integers from 35 to 49 is **630**

ISTITUTO TECNOLOGICO SUPERIORE ACADEMY - INFORMATION AND COMMUNICATIONS TECHNOLOGY
its
ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Defining a function/2

Parameters = Inputs to the function.
"We're expecting 2 parameters and we'll refer to them as `a` and `b`"

Write `def` to signal to the interpreter:
"We're defining a function here!"

```
def functionName(a:int, b:int):
        # code to be executed
        result:int = 0
        for i in range(a, b+1):
                result = result + i

        return result
```

Function body.
Here comes the code that the function will execute.

This indent space is important!
Leaving the indent = leaving the function body.

Question:
What function name
would make sense here?

its ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Using a function

```
def sum(a:int, b:int):

        result:int = 0

        for i in range(a, b+1):
                result = result + i

        return result
```

NOTE

You are ***calling*** the function with the ***arguments*** 1 and 10.

Now, we can just use it:

```
mysum = sum(1, 10)
print(f"Sum from 1 to 10 is {sum(1, 10)}")
```

Try it!

its ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Solution

```
def sum(a:int, b:int):
    # define a sum to be computed
    result:int = 0

    # compute a sum
    for i in range(a,b+1):
        result = result + i

    # return the value of sum as the output of the sum function
    return result



# use sum function to compute a sum of integers from 1 to 10
print(f"Sum from 1 to 10 is  {sum(1,10)}")

# use sum function to compute a sum of integers from 20 to 37
print(f"Sum from 20 to 37 is  {sum(20,37)}")

# use sum function to compute a sum of integers from 35 to 49
print(f"Sum from 35 to 49 is  {sum(35,49)}")
```

ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Functions/1

A *function* is a collection of statements grouped together that performs an operation.

A name is assigned to this group of statements, called function name, which identifies the operation performed by that group of instructions.

Therefore, whenever it is necessary to execute that specific operation, it is sufficient to call the function by its name.

```
# compute sum of integers from 1 to 10
sum(1, 10)

# compute sum of integers from 20 to 37
sum(20, 37)

# compute sum of integers from 35 to 49
sum(35, 49)
```

ISTITUTO TECNOLOGICO SUPERIORE ACADEMY - INFORMATION AND COMMUNICATIONS TECHNOLOGY
its ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Functions/2

- Functions are **blocks of code** that perform a specific task
- They allow you to **break down** your **code** into smaller, **reusable** parts
- Functions can take input, perform operations, and return output
- The code within a function can be **flexible** based on the input (→ **parameters**)

ISTITUTO TECNOLOGICO SUPERIORE ACADEMY - INFORMATION AND COMMUNICATIONS TECHNOLOGY
its ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Functions/3: Exercise

Let's try to define a function named `subtract` ourselves:

- It should take 2 parameters.
- Inside the function, it should subtract the two.
- Then, return the result.

After you defined it, call the function with some arguments!

ISTITUTO TECNOLOGICO SUPERIORE ACADEMY - INFORMATION AND COMMUNICATIONS TECHNOLOGY
its
ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Solution

```
def subtract(a:int, b:int):

        result:int = a - b
        return result



myresult = subtract(4, 1)

print(myresult)

>>> 3
```

User-Defined Functions

All the functions we have used before are functions that we have defined on our own.

This type of function is called a user-defined function.

User-defined functions allow programmers to create reusable blocks of code that perform specific tasks, making the program more organized, readable, and maintainable.

By defining functions, we can break down complex problems into smaller, manageable parts, reducing redundancy and improving efficiency.

Functions can also accept parameters, allowing them to handle different inputs dynamically, and they can return values, making them even more flexible in different contexts.

In summary, user-defined functions are essential for writing structured and efficient code.

ISTITUTO TECNOLOGICO SUPERIORE ACADEMY - INFORMATION AND COMMUNICATIONS TECHNOLOGY
its
ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Function: Return/1

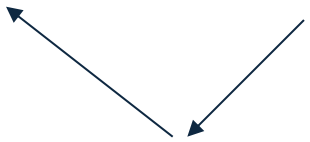
If a function **`return`**s something, it means that that function is “giving” us a value we can “catch”.

```
def subtract(a:int, b:int) -> int:

        result:int = a - b
        return result



myresult = subtract(4, 1)
```



3

```
print(type(subtract(4,1))

>>> <class 'int'>
```

ISTITUTO TECNOLOGICO SUPERIORE ACADEMY - INFORMATION AND COMMUNICATIONS TECHNOLOGY
its ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Return vs. Print

Try to just call our function like so: `subtract(4, 1)`

```
def subtract(a, b):
    result = a - b
    return result
```

```
subtract(4, 1)
>>>
```

We didn't "catch" the return

```
print(subtract(4, 1))
>>> 3
```

We "caught" it and gave it to the print() function

ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Function: Return/2

There are some functions that execute certain instructions without returning any value.
In this case, the return statement is omitted to indicate that the function does not return any value.

```
def greet(name:str) -> None:

        print(f``Hello {name}!'')



greet(“Angela”)


>>> Hello Angela!

Print(type(greet(Angela)))

>>> <class 'NoneType'>
```

ISTITUTO TECNOLOGICO SUPERIORE ACADEMY - INFORMATION AND COMMUNICATIONS TECHNOLOGY
its
ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Function: Return/3

In Python, a function can return **more than one value** using **tuples, lists, or dictionaries**. The most common method is returning a **tuple**, where values are separated by a comma after the return statement.

```
# define a function returning multiple values(returning a tuple)
def operations(a: int, b: int) -> tuple[int, int]:
    sum_result:int = a + b
    diff_result:int = a - b
    # Returns a tuple with two values
    return sum_result, diff_result

# Assigns values to two variables
sum_value, diff_value = operations(10, 5)
print("Sum:", sum_value)                                # Output: Sum: 15
print("Difference:", diff_value)                        # Output: Difference: 5

print(type(operations(10,5)))

>>> <class 'tuple'>
```

ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Function: Return/4

In Python, a function can return **more than one value** using **tuples, lists, or dictionaries**. The most common method is returning a **tuple**, where values are separated by a comma after the return statement.

```
# define a function returning a list
def get_coordinates(x:int, y:int) -> list[float]:
    return [x, y]

coords = get_coordinates(12.5, 45.8)
print(coords[0], coords[1], sep=", ")

>>> 12.5, 45.8

print(type(coords))

>>> <class 'list'>
```

ISTITUTO TECNOLOGICO SUPERIORE ACADEMY - INFORMATION AND COMMUNICATIONS TECHNOLOGY
its ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Function: Return/5

In Python, a function can return **more than one value** using **tuples, lists, or dictionaries**. The most common method is returning a **tuple**, where values are separated by a comma after the return statement.

```
# define a function returning a dictionary
def get_user(myname:str, myrole:str) -> dict[str, str]:
    return {"name": myname, "role": myrole}

user = get_user(“Alice”, “Admin”)
print(user["name"])  # Output: Alice
print(user["role"])  # Output: Admin

print(type(user))

>>> Alice
>>> Admin

>>> <class 'dict'>
```

ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Functions with parameters passed by position/1

There are three ways to pass arguments to parameters:

- pass by position,
- pass by keyword,
- pass by default value.

When arguments are passed by position, the arguments in the calling statement are matched to the parameters in the function header based on their order.

That is,

- the 1st argument is passed to the 1st parameter,
- the 2nd argument is passed to the 2nd parameter, and so on.

```
def describe_person(name:str, age:int, city:str):
    print(f"{name} is {age} years old and lives in {city}.")

# Chiamata alla funzione con passaggio per posizione
describe_person("Alice", 25, "Rome")
```

its ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Functions with parameters passed by position/2

Note: argument order is important!

```
def greet(name:str, age:int) -> None:
    print("Hi, my name is" + name + " and I'm " + str(age) + " years old!")
```

```
greet("Angela", 13)
```

```
>>> Hi, my name is Angela and I'm 13 years old!
```

```
greet(13, "Angela")          # ERROR!
```

Note: Be careful with function parameters!

```
greet()                      # ERROR!
```

its ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Functions with parameters passed by keyword/1

```
def describe_person(name:str, age:int, city:str):
    print(f"{name} is {age} years old and lives in {city}.")

# calling the describe_person function passing values by position
describe_person("Alice", 25, "Rome")
```

Arguments can be passed to functions by using the names of the corresponding parameters instead of relying on position.

This method of passing values to functions is called keyword passing.
For instance, the function `describe_person` can be written

- `describe_person(name="Alice", age=25, city="Rome")`
- `describe_person(age=25, city="Rome", name="Alice")`
- `describe_person(city="Rome", age=25, name="Alice")`
- and so on.

Note: Arguments passed by position must precede arguments passed by keyword. (Similarly for parameters.)

```
describe_person("Alice", city="Rome", age=25)

describe_person(city="Rome", 25, name="Alice") #ERROR! Not Valid!
```

its ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Functions with parameters passed by keyword/2

Note: Arguments passed by position must precede arguments passed by keyword. (Similarly for parameters.)

```
def describe_person(name:str, age:int, city:str):
    print(f"{name} is {age} years old and lives in {city}.")

# calling the describe_person function passing values by position
describe_person("Alice", 25, "Rome")
```

```
describe_person("Alice", city="Rome", age=25)
```

```
describe_person(city="Rome", 25, name="Alice") #ERROR! Not Valid!
```

its ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Functions with default values/1

Some (or all) of the parameters of a function can have **default values**—values that are assigned to them when no values are passed as input arguments.

That is, if the corresponding arguments are omitted when the function is called, the default values are assigned to the parameters.

A typical format for a function definition using default values is

```
def functionName(par1, par2, par3=value3, par4=value4):
```

Then, a function call of the form functionName(*arg1*, *arg2*) will assign the value of *arg1* to *par1*, the value of *arg2* to *par2*, *value3* to *par3*, and *value4* to *arg4*. The assignment of values to par3 and par4 is optional.

In a function definition, the parameters without default values must precede the parameters with default values.

For instance, a header such as

```
def func(par1, par2=value2, par3):
```

is not valid.

ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Functions with default values/2

For instance, consider the function *total* defined by

```
def total(w, x, y=10, z=20):
      return (w ** x) + y + z

# calling function total, while omitting the last two values
total(2, 3)
# Output: 38 ->  calulated as 2^3 + 10 + 20


# calling function total, while omitting the last values
total(2, 3, 4)
# Output: 32 -> calculated as 2^3 + 4 + 20


# calling function total with 4 input values
total(2, 3, 4, 5)
# Output: 17 -> calculated as 2^3 + 4 + 5
```

ISTITUTO TECNOLOGICO SUPERIORE ACADEMY - INFORMATION AND COMMUNICATIONS TECHNOLOGY
its ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Functions without parameters/1

Some functions can be defined and executed without receiving any input values or parameters.

In Python, functions do not always require arguments to perform an action. These types of functions are useful when a task needs to be executed in a standard way, without depending on external input.

```
def hello():

        print(‘‘Hello’’)



hello()


>>> Hello
```

ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Functions without parameters/2

Even if some functions do not receive any input parameters, they can still return a value.

The decision of whether a function should accept parameters or return a value entirely depends on the task it needs to accomplish. If a function needs to return a value, it means that its output will be used elsewhere in the program, for example, to assign it to a variable or pass its result to another function.

On the other hand, a function may not receive any input parameters and still perform useful operations, such as returning a predefined value or working with global variables.

```
def get_pi():
    return 3.14159

pi_value:float = get_pi()
print("The value of pi is:", pi_value)

>>> The value of pi is:3.14159
```

its ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Functions, parameters and return

- If the function's result needs to be used in other parts of the program, it is useful to return a value.
- If the function needs to work with external data provided by the user or the program, it is useful to accept parameters.
- If the function only performs an action (such as printing a message), it may not need either parameters or return values.

Ultimately, the structure of a function depends on the purpose it needs to achieve and the efficiency with which we want to organize our code.

ISTITUTO TECNOLOGICO SUPERIORE ACADEMY - INFORMATION AND COMMUNICATIONS TECHNOLOGY
its
ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Functions calls and execution/1

To use a function, you have to ***call*** or ***invoke*** it.

When a program calls a function, program control is transferred to the called function.

A called function returns control to the caller when its return statement is executed or the function is finished.

How is this program executed?

- The interpreter reads the script in the file line by line starting from line 1.
- When it reads the function header in line 1, it stores the function with its body (lines 1–5) in the memory.
- Remember that a function's definition defines the function, but it does not cause the function to execute.

```
1  def max(num1:int, num2:int) -> int:
2      if num1 > num2:
3          return num1
4      else:
5          return num2
6
7  # calling max function
8  numMax:int = max(3,4)
9  print(numMax)
```

ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Functions execution/2

- When the **max** function is invoked (line 8), value **3** is passed to **num1** and value **4** is passed to **num2**.
- The control is transferred to the **max** function, and the **max** function is executed.
- When the **return** statement in the **max** function is executed, the **max** function returns the control to its caller.
- After the **max** function is finished, the returned value from the **max** function is assigned to **numMax** (line 8).
- Code prints the result (line 9).
- The program is now finished.

```
1  def max(num1:int, num2:int) -> int:
2      if num1 > num2:
3          return num1
4      else:
5          return num2
6
7  # calling max function
8  numMax:int = max(3,4)
9  print(numMax)
```

its ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Functions execution/3

- Before calling max(3,4) , the control is in the program (at the instruction numMax = max(3,4)).
- During the execution of the function, the control passes to the function max.
- When the function max finishes, the control returns to its caller, which is the instruction numMax = max(3,4).
- In conclusion, the **caller** of the function max is the instruction numMax = max(3,4), which is part of the **program**.

```python
def max(num1:int, num2:int) -> int:
    if num1 > num2:
        return num1
    else:
        return num2

# calling max function
numMax:int = max(3,4)
print(numMax)
```

ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Call Stacks

Each time a function is invoked, the system creates an *activation record* that stores its arguments and variables for the function and places the activation record in an area of memory known as a *call stack.*

When a function calls another function, the caller's activation record is kept intact and a new activation record is created for the new function call.

When a function finishes its work and returns control to its caller, its activation record is removed from the call stack.

A call stack stores the activation records in a **last-in, first-out** fashion.

The activation record for the function that is invoked last is removed first from the stack.

Suppose function **m1** calls function **m2**, and then **m3**.

- The runtime system pushes **m1**'s activation record into the stack, then **m2**'s, and then **m3**'s.
- After **m3** is finished, its activation record is removed from the stack.
- After **m2** is finished, its activation record is removed from the stack.
- After **m1** is finished, its activation record is removed from the stack.

ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Functions and args/1

How can we create functions that accept a variable number of arguments?
Imagine you want to create a function that adds two numbers:

```
def add(a, b):
    return a + b

print(add(2, 3))      # Works!

print(add(2, 3, 4))   # ERROR: Too many arguments!
```

As we can see, if we pass more than two numbers, the function gives an error!

So, we need to use ***args** to accept any number of values!

its ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Functions and args/2

In general, if we have a list called **`mylist`**, in the statement `print(mylist)`, the list `mylist` is treated as a single object. This means that the print function receives a single input argument, which is the list itself (`mylist`).

On the other hand, the expression **`*mylist`** consider each individual element of the list separately.
Therefore, in the statement `print(*mylist)`, each element of the list `mylist` is extracted and passed as a separate argument to the print function. As a result, the elements are printed individually in output, rather than a single list.

Similarly, when defining a function, **`*args`** behave in the same way.
`args` is a list of values inside the function. So, using **`*args`** allows us to pass an unlimited number of values to a function.
Just like `*mylist` unpacks elements of my list when printing, **`*args`** unpack input arguments when calling a function, making it more flexible and dynamic.

ISTITUTO TECNOLOGICO SUPERIORE ACADEMY - INFORMATION AND COMMUNICATIONS TECHNOLOGY
its ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Functions and args/3

```
def add(*args):  # Accepts multiple numbers in input

    total = 0
    for number in args:
        total += number
    return total


print(add(2, 3))          # Output 5

print(add(10, 20, 30))    # Output 60
```

ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Functions and kwargs/1

But what if we want named arguments?
`*args` is great for lists of values, but what if we need to pass information **with specific names**?

Example: We want to calculate the total price of products (coffee, cake, juice). If we use only `*args`, we lose the product name:

```
def total_price(*args):
    total:float = sum(args)
    return total

print(total_price(2.99, 4.55, 2.99))  # Works, but what are these prices?
```

We don't know which price belongs to which product!

So, we need to use **`**kwargs`** to pass key-value pairs (name and price)

ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Functions and kwargs/2

`**kwargs` allows us to pass **an unlimited number of named arguments**, like a dictionary.
Example of a function that calculates the total price of an order, with product names:

```
def total_price(**kwargs):
    total:float = 0
    for product, price in kwargs.items():
        print(f"{product}: {price}€")
        total += price
    return round(total, 2)

print(total_price(coffee=2.99, cake=4.55, juice=2.99))
```

```
>>>
coffee: 2.99€
cake: 4.55€
juice: 2.99€
Total: 10.53€
```

Now we know exactly what products we are adding up!
is a dictionary of key-value pairs inside the function.

ISTITUTO TECNOLOGICO SUPERIORE ACADEMY - INFORMATION AND COMMUNICATIONS TECHNOLOGY
its ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Built-in Functions/1

Python comes with a variety of built-in functions that are readily available for use.
Built-in functions are like miniature programs. They receive input, they process the input and they produce output.

Examples of built-in functions include `print()`, `len()`, `max()`, `min()`, `sum()`, `abs()`, `round()`, `sorted()`, `range()`, and many more.

To use a built-in function, simply call it by its name followed by parentheses.

Example: `print("Hello, world!")`

ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Built-in Functions/2

Common built-in functions:

- `print()`: Outputs data to the console.
- `len()`: Returns the length of an object (e.g., a string or list).
- `max()`: Returns the largest item in an iterable.
- `min()`: Returns the smallest item in an iterable.
- `sum()`: Returns the sum of all items in an iterable.
- `abs()`: Returns the absolute value of a number.
- `round()`: Rounds a number to a specified number of decimal places.
- `sorted()`: Returns a new sorted list from the elements of an iterable.
- `range()`: Generates a sequence of numbers.

its ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Let’s get acquainted with common functions

- `len()`: Returns the length of an object (e.g., a string or list).

- Define a list and fill it with elements
- Save the length of the list in a variable `list_len`
- Print `list_len`

```
mylist = [“this”, “is”, “my”, “list”]
list_len = len(mylist)
print(list_len)
>>> 4
```

its ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Let’s get acquainted with common functions

- `max()`: Returns the largest item in an iterable.

- Define a list and fill it with numbers.
- Save the maximum element of the list in a variable `max_element`
- Print `max_element`

```
mylist = [9, 4, 2, 8]
max_element = max(mylist)
print(max_element)
>>> 9
```

ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Let’s get acquainted with common functions

- `sum()`: Returns the sum of all items in an iterable.

- Define a list and fill it with numbers.
- Save the sum of the list in a variable `list_sum`
- Print `list_sum`

```
mylist = [9, 4, 2, 8]
list_sum = sum(mylist)
print(list_sum)
>>> 23
```

its ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Let’s get acquainted with common functions

- `sorted()`: Returns a new sorted list from the elements of an iterable.

- Define this list: [5, 2, 9, 1]
- Save the sorted version of the list in `sorted_list`
- Print `sorted_list`

```
mylist = [5, 2, 9, 1]
sorted_list = sorted(mylist)
print(sorted_list)
>>> [1, 2, 5, 9]
```

its ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY